

TurboRipassi

Documentazione di progetto

App personale per ripassi programmati con ripetizione temporale

Nikita Piraino

(codice interamente generato da AI, con supervisione: vedi il capitolo “Metodo di lavoro”)

Revisione 3 — 3 agosto 2026

Stato dei documenti del progetto. Questo documento descrive lo *stato attuale* del sistema ed è la fonte autorevole in caso di divergenza. `ripassa-app-spec.md` raccoglie i *requisiti iniziali* (luglio 2026) ed è conservato come documento storico: gli scostamenti consapevoli rispetto ad esso sono elencati nella sezione “Scostamenti dalla specifica iniziale”. `README.md` e `BUILD.md` contengono le istruzioni operative.

Indice

I	La soluzione, in parole semplici	4
1	Il problema	5
2	La soluzione: TurboRipassi	6
2.1	Cosa fa	6
2.2	Come si usa	6
2.3	Le decisioni di fondo	7
3	Il redesign: da app singola a compagno di corso	8
3.1	Che cosa è cambiato	8
3.2	Le tre regole del sistema di design	8
3.3	Che cosa è vero e che cosa è una dimostrazione	9
4	Cosa vogliamo realizzare	10
4.1	Stato attuale (fatto)	10
4.2	Prossimi passi	10
4.3	Possibili sviluppi futuri (oggi fuori scopo)	11
II	Documentazione tecnica	12
5	Stack e vincoli	13
6	Architettura	14
6.1	Le regole, e come sono verificate	14
6.2	Contratti fra i livelli	15
6.3	Mappa del codice	16
7	Modello dati	18
7.1	ripassi	19
7.2	occorrenze	19
7.3	allegati	19
7.4	cache_allegati (solo SQLite on-device)	19
8	Sincronizzazione cross-device	21
9	Cache locale e rotazione	22
9.1	Un fallimento silenzioso non è più silenzioso	22
10	Sicurezza	24
10.1	Autenticazione	24
10.2	Account e identità di accesso	25

10.3 Row Level Security	26
10.4 I binari degli allegati	26
11 Dove vivono gli allegati	28
11.1 Il vincolo che ha forzato la scelta	28
11.2 Perché non riporta il problema di partenza	28
11.3 Cosa cambia, in concreto	29
12 Ciclo di vita di un allegato	30
12.1 Caricamento	30
12.2 Apertura	30
12.3 Rinomina, riordino, eliminazione	31
13 Affidabilità in rete	32
13.1 Errori comprensibili	32
13.2 Offline esplicito	33
13.3 Retry: dove sì e dove no	33
13.4 Ordinamento totale	33
14 Crash reporting e resilienza	34
15 Test e qualità	35
15.1 La seconda suite: i test SQL	35
15.2 Copertura per livello	36
15.3 Che cosa i test verificano davvero	36
15.4 Limiti noti di copertura	36
15.5 Controlli statici	37
16 Build e distribuzione	39
16.1 Aggiornamento dello schema	39
16.2 Fase 0 — sviluppo (Expo Go)	39
16.3 Sicurezza delle dipendenze	39
16.4 Fase 1 — app installate (vedi BUILD.md)	39
17 Scostamenti dalla specifica iniziale	41
18 Metodo di lavoro e contributo personale	43
18.1 Il ciclo effettivo	43
18.2 Che cosa ha trovato la verifica	43
18.3 Le quattro revisioni	44
18.4 Limite dichiarato del metodo	45
19 Rischi aperti	46

Parte I

La soluzione, in parole semplici

Capitolo 1

Il problema

Chi studia con il metodo della *ripetizione programmata* rivede lo stesso argomento a intervalli crescenti: il giorno dopo, dopo una settimana, dopo un mese, dopo sei mesi. Ogni “ripasso” ha bisogno del suo materiale: le foto degli appunti, i PDF delle dispense, qualche nota scritta al volo.

L’app usata finora per questo scopo (“Genio in 21 Giorni”) ha il concetto giusto ma un difetto strutturale: su iPad salva i dati su iCloud, su Android su Google Drive, e i due mondi non si parlano. Il risultato è che il materiale di studio è frammentato tra dispositivi, e quello che si aggiunge da un tablet non si ritrova sul telefono.

Capitolo 2

La soluzione: TurboRipassi

TurboRipassi è un'app personale, pensata per un solo utente, che fa una cosa sola e la fa su tutti i dispositivi insieme: **Android (Samsung), iPad e Mac vedono sempre gli stessi ripassi, le stesse date, gli stessi allegati**, grazie a un unico account e a un unico archivio centrale (Supabase).

2.1 Cosa fa

- **Crea un ripasso** con titolo e note libere.
- **Genera da sola le date di ripasso**: +1 giorno, +1 settimana, +1 mese, +6 mesi dal momento della creazione. Un interruttore (spento di default) aggiunge anche un ripasso “+1 ora” per il ripasso immediato.
- **Allega materiale**: foto scattate al momento, immagini dalla galleria, PDF e altri file. Gli allegati si possono rinominare, riordinare, eliminare.
- **Sincronizza in tempo reale**: una modifica fatta dal telefono compare sull'iPad senza fare nulla.
- **Funziona anche senza rete** (in treno, in metro): l'app tiene in locale gli allegati dei ripassi di *ieri, oggi e domani*, scaricandoli in anticipo e cancellando dal dispositivo quelli che non servono più. Il materiale resta comunque sempre al sicuro nel cloud.

2.2 Come si usa

Dopo il login l'app si apre su una barra di quattro schede — *Home, Allenati, TurboRipassi, Contenuti* — con un menu laterale per il resto (programma di studio, appuntamenti, corsi, obiettivi, flashcard, profilo). Il redesign che ha portato le quattro schede è descritto al capitolo 3. Le schermate che riguardano i ripassi, cioè quelle di cui parla il resto di questa relazione, sono quattro:

1. **Accesso** — una volta sola per dispositivo, con l'account Google oppure con email e password. Da lì in poi la sessione resta attiva.
2. **TurboRipassi** — l'elenco dei ripassi programmati, una riga per data, raggruppati per scadenza (in ritardo, oggi, questa settimana, più avanti) e in due schede: *Da ripassare* e *Storico* (i giorni già passati, con un filtro per vedere solo quelli mai completati). Il tondino a sinistra del titolo segna la singola data come fatta. In alto la ricerca, la striscia “Come funziona?” e il pulsante per aggiungerne uno.
3. **Scheda ripasso** — titolo, note, l'elenco delle date programmate (ognuna si può segnare come completata, posticipare o anticipare) e l'accesso agli allegati.

4. **Allegati** — la galleria del materiale: si apre a schermo intero, si rinomina, si riordina, si elimina.

2.3 Le decisioni di fondo

- **Un solo account per tutto:** il problema dell'app precedente non deve ripresentarsi. Il difetto di “Genio in 21 Giorni” non era usare Google Drive, era usare *archivi diversi su piattaforme diverse* (iCloud su iPad, Drive su Android). Gli allegati oggi stanno su un unico Google Drive, lo stesso su Android, iPad e Mac: la scelta e le sue conseguenze sono documentate nel capitolo “Dove vivono gli allegati”.
- **Tutto gratuito** dove possibile; unica eccezione accettabile un costo minimo (~2 €/mese) se davvero necessario.
- **Niente statistiche, grafici o notifiche push:** fuori scopo, per scelta esplicita. I promemoria che l'app manda quando scade un ripasso sono notifiche *locali*, pianificate sul dispositivo: non c'è nessun servizio push, nessun token e nessun server che decida quando parlare all'utente.
- **Codice scritto interamente da AI**, supervisionato ma non scritto a mano: lo stack è stato scelto anche per essere ben documentato e “generabile” in modo affidabile.
- **App nativa**, non sito web: serve accesso a fotocamera, file e archiviazione locale per l'uso offline.

Capitolo 3

Il redesign: da app singola a compagno di corso

L'ultima revisione è anche la più grande in volume di codice, e fino a questa stesura non era documentata da nessuna parte. Ha cambiato che cosa l'app è: da uno strumento che fa una cosa sola a un compagno del corso “Genio in 21 giorni”, dentro cui i ripassi programmati diventano una scheda su quattro. La ragione non è estetica ed è onesto scriverla: l'app è proponibile a chi tiene il corso solo se parla la lingua del corso, e un elenco di date non la parla.

3.1 Che cosa è cambiato

- **Quattro schede** al posto di una schermata: *Home* (dove sono nel percorso), *Allenati* (gli esercizi del metodo), *TurboRipassi* (l'app precedente, per intero e senza perdite), *Contenuti* (materiale da leggere e vedere). Il resto — programma, appuntamenti, corsi, obiettivi, flashcard, profilo — sta in un menu laterale, perché sono cose che si aprono raramente e una scheda è un impegno permanente sullo schermo.
- **Un sistema di componenti** (`view/components/organic/`) al posto delle *View* grezze. Prima ogni schermata disegnava la propria idea di scheda, con il proprio raggio e il proprio padding, e due blu venivano usati senza un ruolo. Adesso le forme sono nominate — scheda, chip, pillola, tacche, batteria, segmentato — e “la stessa cosa” si assomiglia per costruzione invece che per revisione.
- **Quattro fasi** (ospite, pre, durante, post), calcolate nel Model da una sola data di iscrizione. La Home ha una variante per ciascuna, e nessuna schermata calcola la fase per conto proprio: la calcola `PercorsoContext`, una volta, e le altre la ricevono. Quattro schermate che se la calcolassero da sole finirebbero, prima o poi, per non essere d'accordo su che giorno è.
- **Il velo lead-gen**: chi non è iscritto vede le schermate del percorso attraverso una scheda scura che spiega che cosa otterrebbe. Non è un blocco tecnico ed è giusto dirlo — è una scelta di prodotto, e il dato sotto non è segreto.

3.2 Le tre regole del sistema di design

1. **Una forma per concetto.** Se due cose si disegnano uguali sono la stessa cosa. Il corollario è che la scheda scura è riservata ai quattro o cinque momenti in cui l'app parla in prima persona: se “scuro” diventasse un'opzione grafica, smetterebbe di significare qualcosa.
2. **Il maiuscolo in un posto solo**, il *kicker* sopra una sezione. L'app precedente urlava titoli interi (“SCEGLI IL TUO PROGRAMMA DI STUDIO”): costa leggibilità e non

costruisce nessuna gerarchia. Dieci pixel di maiuscoletto spaziato sopra un titolo in tondo la costruiscono.

3. **I colori hanno un ruolo, non un nome.** Nessun componente conosce un esadecimale: chiedono a `theme` l'accento, l'inchiostro su fondo scuro, il bordo. È questa regola ad aver reso possibile l'ultima modifica di questa revisione — il ritorno alla palette blu/gialla del TurboRipassi precedente è stato un solo file, `theme.ts`, più cinque valori letterali rimasti in giro. Struttura, spaziature e tipografia del redesign sono rimaste dov'erano: è cambiata la voce, non la grammatica.

3.3 Che cosa è vero e che cosa è una dimostrazione

La distinzione conta più della grafica, ed è la ragione per cui questo capitolo esiste.

- **Vero:** lo stato del percorso. Iscrizione, padronanze raggiunte, lingua del mazzo e programma scelto sono dati che l'utente produce, che sopravvivono alla chiusura dell'app e che passano da un Controller (`PercorsoContext`) e da un repository con interfaccia (`PercorsoRepo`).
- **Dimostrazione:** allenamenti, contenuti, flashcard e agenda. Sono cataloghi costanti nel Model, senza API e senza scrittura. Le schermate che li mostrano non hanno un hook di Controller, e va bene finché il dato è statico — è anche il motivo per cui i quattro moduli sono ammessi esplicitamente nella lista dei moduli puri che la View può importare, invece di passare inosservati.
- **Il prezzo,** dichiarato anche nel capitolo sugli scostamenti: lo stato del percorso vive in un file sul dispositivo e *non* si sincronizza. È l'unica eccezione al principio fondativo “un solo account, le stesse cose su ogni dispositivo”. Come toglierla è scritto: `supabase/futuro/0004_percorso.sql` contiene schema e policy RLS già pronti, e `percorsoRepo.ts` contiene l'implementazione remota che li userebbe. Nessuno dei due è attivo, per la ragione detta al capitolo sui prossimi passi.

Capitolo 4

Cosa vogliamo realizzare

4.1 Stato attuale (fatto)

L'MVP è completo e funzionante: login (Google e email/password), creazione ripassi con date automatiche, allegati sul Google Drive dell'utente con compressione delle foto, sincronizzazione in tempo reale, cache offline con rotazione automatica, ricerca, storico. Il codice è coperto da una suite di test automatici (logica di dominio, repository, hook del Controller e una schermata; conteggio e copertura nel cap. 15) e ha superato un audit di sicurezza; `lint`, typecheck e test girano a ogni push in integrazione continua. Dettagli nella parte tecnica.

Nota sull'esecuzione: l'autorizzazione a Google Drive richiede uno schema di redirect derivato dall'`applicationId`, che Expo Go non può onorare. Lo sviluppo quotidiano resta possibile con Expo Go per tutto il resto, ma **gli allegati funzionano solo su una build installata** (development build o APK): è la ragione principale per cui la "Fase 1" non è più opzionale.

È stata inoltre predisposta la rete di sicurezza necessaria per distribuire l'app a persone diverse dallo sviluppatore: se qualcosa va storto sul dispositivo di un utente, l'errore viene segnalato automaticamente (Sentry) e l'app mostra una schermata di errore con un pulsante "Riprova" invece di chiudersi o restare bianca. Il servizio va attivato inserendo una chiave di configurazione al momento della distribuzione: senza, l'app funziona comunque, semplicemente non segnala nulla.

4.2 Prossimi passi

1. **App installate stabilmente** ("Fase 1"): un APK per il Samsung costruito nel cloud di Expo (gratis), e l'app per iPad firmata dal Mac con l'Apple ID gratuito. Con l'app installata il login persiste e non serve più Expo Go né la rete del Mac. La procedura completa è in `BUILD.md`.
2. **Validazione sul Mac M2**: verificare che l'app per iPad giri nativamente sul Mac come app "Designed for iPad" (rischio principale individuato dalla specifica).
3. **Decisione consapevole sul rinnovo settimanale**: con l'Apple ID gratuito la firma dell'app iPad scade ogni 7 giorni e va rinnovata ricollegando l'iPad al Mac. L'alternativa è l'Apple Developer Program (99 \$/anno). Si parte gratis e si valuta sull'uso reale.
4. **Attivare il crash reporting**: creare il progetto su `sentry.io` e registrare la chiave (`EXPO_PUBLIC_SENTRY_DSN`) su EAS. Il codice è già pronto: manca solo la chiave. Passaggio necessario prima di dare l'app a utenti reali, altrimenti i loro crash restano invisibili.

4.3 Possibili sviluppi futuri (oggi fuori scopo)

Non fanno parte dell'MVP, ma l'architettura non li preclude: uso multi-utente (le protezioni dati per-utente sono già attive sul server, in previsione di proporre l'app a terzi), pubblicazione sugli store, notifiche push.

Il percorso su Supabase: specificato, non fatto, e sospeso a una condizione. Lo stato del percorso è l'unica cosa che due dispositivi dello stesso account vedono diversa (cap. 3). La soluzione è scritta per intero e non è stata applicata: `supabase/futuro/0004_percorso.sql` contiene la tabella (`user_id` come chiave primaria, lo stato in `jsonb`, una riga per account) e la policy RLS `percorso_owner` sul modello di `ripassi_owner`; il commento in testa a `percorsoRepo.ts` contiene l'implementazione remota di `PercorsoRepo` e le tre righe che la metterebbero in funzione. Il file sta in `supabase/futuro/` e non in `supabase/migrations/` proprio per non dare l'impressione che sia attivo: nessun progetto lo ha eseguito e nessuna riga di TypeScript lo legge.

La condizione è esplicita: **si farà se e solo se il prodotto verrà venduto ufficialmente a Genio in 21 giorni**. Finché il percorso è una dimostrazione del redesign, un file locale è la scelta proporzionata — il dato si ricostruisce in pochi tocchi, e una tabella in più sul server sarebbe uno schema da mantenere per una funzione che nessuno ha comprato. Scriverla adesso serve a un'altra cosa: la forma della soluzione ha deciso la forma del codice. `PercorsoRepo` è un'interfaccia iniettata nel provider, e non due funzioni importate, perché quella tabella possa sostituire il file senza che il Controller se ne accorga. Il lavoro vero, del resto, non sono le due query: è decidere che cosa fare offline, cioè usare il file come cache del server esattamente come `localCache` fa per i ripassi.

La **scrittura offline** era in questo elenco e non ci è più: è stata realizzata. Un ripasso creato senza rete finisce in una coda su file (`model/outbox/`), con una copia privata degli allegati in attesa di salire su Drive; la lista dice quanti elementi devono ancora essere caricati e la coda si svuota da sola quando la connessione torna. Il rientro nello scopo è una decisione, non una dimenticanza: senza di essa un ripasso creato in metropolitana andava perso, cioè proprio il caso d'uso che aveva motivato la cache offline in lettura.

Parte II

Documentazione tecnica

Capitolo 5

Stack e vincoli

Livello	Scelta	Motivo
Frontend	React Native + Expo SDK 54	un codebase per Android/iPadOS/macOS
Dati e sync	Supabase (Postgres, Realtime, Auth)	gratuito per uso personale, realtime nativo, un solo account
Allegati (binari)	Google Drive dell'utente (scope <code>drive.file</code>)	niente tetto da 1 GB; vedi cap. 6
Cache locale	expo-sqlite + FileSystem	lettura offline senza servizi esterni
Autenticazione	Supabase Auth	Google OAuth (PKCE) + email/password
Crash reporting	Sentry (@sentry/react-native)	visibilità sui crash delle build distribuite; disattivabile senza DSN
Test	jest-expo + Testing Library RN	logica, repository e hook testabili senza dispositivo
Qualità	ESLint + tsc strict + CI	confini di architettura verificati

Limiti del piano gratuito Supabase (verificati, luglio 2026): 500 MB database, 1 GB storage file, 5 GB/mese di banda in uscita, pausa automatica del progetto dopo 7 giorni senza richieste. Il tetto di 1 GB è stato *il* vincolo dimensionante del progetto ed è la ragione per cui i binari degli allegati non stanno più su Supabase (capitolo 6). Su Postgres restano solo i metadati, che a 500 MB non sono un problema realistico. Le immagini vengono comunque compresse prima dell'upload (ridimensionamento a 1600px, JPEG 70%), perché lo spazio consumato ora è quello del Drive personale dell'utente.

Capitolo 6

Architettura

Separazione **Model / Controller / View** obbligatoria, più un livello trasversale **config** che nella specifica iniziale non era previsto e che il codice ha reso necessario: l'infrastruttura (client Supabase, SDK Sentry, storage cifrato, lettura della configurazione) non appartiene al dominio, ma serve a tutti e tre i livelli.

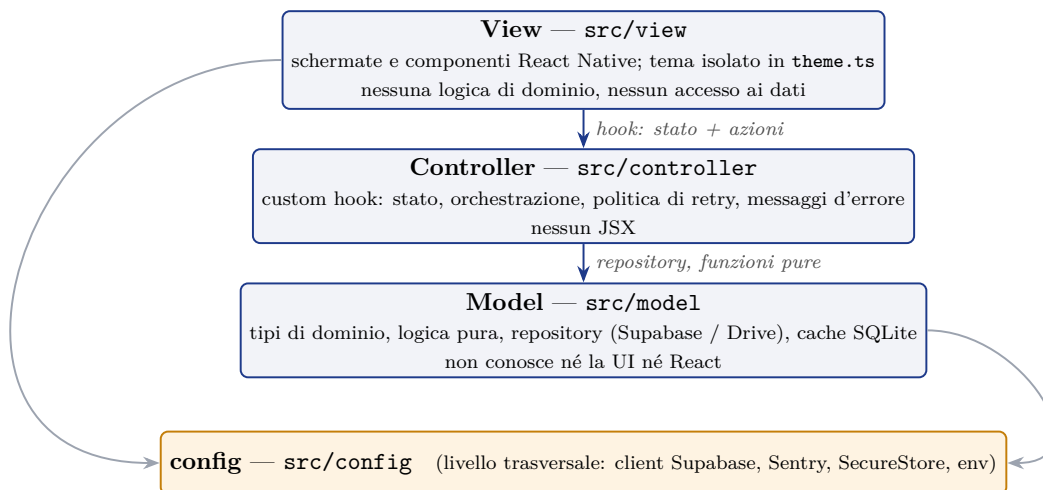


Figura 6.1: I livelli e le sole direzioni di dipendenza consentite. Le frecce grigie indicano che ogni livello può importare **config**, mentre **config** non importa nessuno di essi.

6.1 Le regole, e come sono verificate

1. Nessuna View importa il client Supabase, un repository, la cache o il client Drive: ogni accesso ai dati passa da un hook del Controller. Restano leciti dalla View i *tipi* di dominio e le funzioni pure (`ripassiLogic`, `fileUtils`), che non sono I/O.
2. Il Model non importa né View né Controller: le dipendenze puntano solo verso il basso.
3. Il Controller non importa la View.
4. **config** non importa nessun livello applicativo. Questa regola non è teorica: il gestore dei token Drive stava in **config** e importava dal Model, creando un ciclo; è stato spostato in `model/drive/`.

Le quattro regole sono eseguibili. Fino alla revisione precedente vivevano solo in questa pagina, ed erano già state violate una volta e ripristinate a mano durante l'audit. Ora sono scritte in `eslint.config.js` come `no-restricted-imports` per zona, con un messaggio che

spiega la regola violata: `npm run lint` fallisce, in locale e in CI. Una regola di architettura che nessuno può infrangere per distrazione vale più di una regola scritta bene.

Perché la prima regola è scritta al positivo. Fino alla revisione precedente la regola sulla View elencava i *divieti*: `config/supabase`, i file `*Repo`, la cache, il client Drive. Era una lista di percorsi e non una definizione di “accesso ai dati”, e una lista di divieti fallisce aperta — un modulo di dati che non si chiama `*Repo` ci passa accanto senza toccarla. È successo davvero: i cataloghi arrivati col redesign (`model/percorso`, `model/contenuti`, `model/flashcard`) sono entrati nelle schermate senza che la regola avesse niente da dire, e la lacuna è stata trovata da una rilettura, non dal linter.

Oggi la regola è rovesciata: `@/model/**` è vietato alla View, e l’unica cosa enumerata è la lista `MODEL_PURO` dei moduli ammessi (tipi, cataloghi costanti, funzioni dei propri argomenti). Un modulo nuovo del Model è vietato per difetto, e ammetterlo costa una riga in `eslint.config.js`, dove si vede in revisione. La verifica è diretta: un import di `percorsoRepo` o `ripassiRepo` da un file della View fa fallire `npm run lint`, un import di `percorso/fasi` no.

Resta un limite, e non è più il linter. Le quattro schermate del percorso leggono cataloghi statici senza un hook in mezzo: alla lettera è lecito, ma è lecito perché quei dati oggi non vengono da nessuna parte. Il giorno in cui arriva un’API sono le uniche che dovranno cambiare forma e non solo sorgente. Lo stato del percorso, che invece è già stato vero, passa regolarmente da un Controller (`PercorsoContext`) e da un repository con interfaccia (`PercorsoRepo`).

6.2 Contratti fra i livelli

Ogni livello espone un’interfaccia dichiarata, non inferita dall’implementazione.

Confine	Contratto
Model → Controller	<code>RipassiRepo</code> , <code>AllegatiRepo</code> , <code>DriveClient</code> , <code>DriveTokenManager</code> , <code>NotificheRepo</code> , <code>PercorsoRepo</code> : interfacce con un’implementazione di default. Gli hook le ricevono come parametro con valore predefinito, quindi un test passa un oggetto finto senza toccare il sistema dei moduli.
Controller → View	<code>StatoAuth</code> , <code>StatoRipassi</code> , <code>StatoCache</code> , <code>ContestoRipassi</code> , <code>ContestoPercorso</code> : interfacce esportate e annotate come tipo di ritorno degli hook, così una divergenza fallisce nel file dell’hook e non nelle schermate che lo consumano.
Aree senza contratto	Allenamenti, contenuti, flashcard e agenda: la View legge il catalogo dal Model senza un hook in mezzo, perché non c’è nessuno stato da possedere finché il dato è statico. È la lacuna dichiarata sopra, ed è l’unica rimasta: sono i quattro moduli ammessi esplicitamente in <code>MODEL_PURO</code> e non un’omissione.

Gestione degli errori: chi lancia, chi traduce, chi parla

È una politica di livello, uniforme in tutto il codice:

- Il **Model** lancia l’errore originale, senza tradurlo e senza registrarlo: `if (error) throw error`. Non sa chi lo userà.
- Il **Controller** decide. Traduce con `traduciErrore` (Model), sceglie se ritentare con `conRetry` (Model) e segnala con `mostraErrore`, unico canale verso l’utente, che a sua volta registra l’evento su Sentry.
- La **View** non *traduce* errori e non decide se ritentare: mostra la stringa che riceve. Può però *consegnarne* una, chiamando `mostraErrore` — succede in otto punti fra `RipassiScreen`,

`FormRipassoScreen` e `DettaglioAllegatiScreen`, per gli errori che nascono nella schermata stessa (un file troppo grande scelto dal selettore, una conferma rifiutata) e che nessun hook ha mai visto. La regola che conta è che il canale resti uno solo, non che la View non lo usi: se aprisse un `Alert` suo, l'evento non arriverebbe a Sentry.

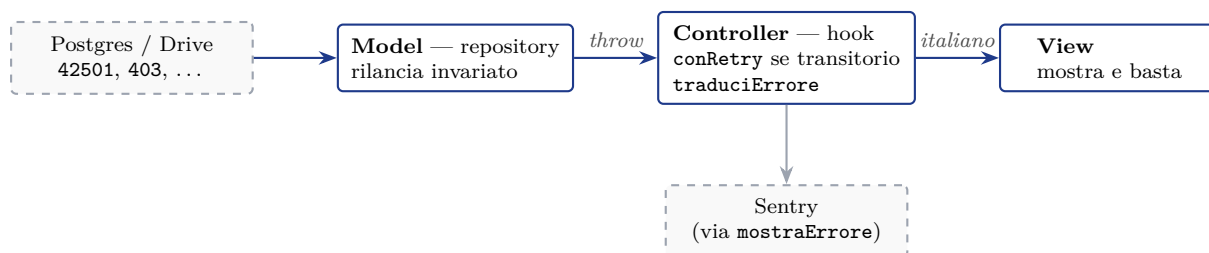


Figura 6.2: Percorso di un errore tecnico fino all'utente. Il testo originale non supera mai il Controller, tranne nell'unico caso documentato al capitolo 13.1.

6.3 Mappa del codice

Il codice è organizzato per *livello* e, dentro ogni livello, per *dominio* (`ripassi`, `allegati`, `drive`, `cache`, `auth`), così che una modifica funzionale tocchi una cartella e non tre.

File	Responsabilità
Radice	
App.tsx	gating autenticazione, navigazione stack, ErrorBoundary, init crash reporting
config/ — infrastruttura trasversale	
env.ts	unico lettore di configurazione: env → app.json/extra , riconoscimento placeholder
supabase.ts	unica istanza del client; PKCE; refresh token via AppState
secureAuthStorage.ts	sessione cifrata in Keychain/Keystore (a blocchi da 1800 byte)
crashReporting.ts	unico punto di accesso a Sentry: init , DSN , reportError
driveConfig.ts	client ID per piattaforma, scope drive.file , cartella applicativa
model/ — dominio, dati, I/O	
types.ts	tipi del dominio (specchiano lo schema Postgres)
ripassi/occorrenzeDates.ts	calcolo puro delle date (+1h/+1g/+1s/+1m/+6m), comparatore
ripassi/ripassiLogic.ts	classificazione per giornata (ripassi/storico), ordinamento totale, ricerca
ripassi/ripassiRepo.ts	RipassiRepo : CRUD ripassi/occorrenze su Supabase
allegati/allegatiRepo.ts	AllegatiRepo : upload su Drive + metadati, riordino, eliminazione
drive/driveTypes.ts	interfacce ed errori tipizzati del backend Drive
drive/driveRepo.ts	client REST Drive: cartella, upload in due passi, download, delete
drive/driveAuth.ts	OAuth Drive con PKCE + refresh token, persistiti in SecureStore
cache/cacheLogic.ts	logica pura della rotazione (finestra, selezione, eliminazione)
cache/localCache.ts	I/O cache: SQLite cache_allegati + filesystem
auth/oauthRedirect.ts	parsing e confronto degli URL di redirect
auth/codiciUsati.ts	codici OAuth monouso già spesi, condivisi dai due flussi
ripassi/ripassiOffline.ts	ultima lista conosciuta su file, per la partenza senza rete
outbox/codaLogic.ts	regole pure della coda: cosa va sincronizzato, come appare
outbox/coda.ts	coda su disco + copie private dei file in attesa di Drive
notifiche/notificheLogic.ts	quali occorrenze meritano un promemoria, e quando
notifiche/notificheRepo.ts	NotificheRepo : pianificazione locale per id di occorrenza
percorso/fasi.ts	fase (ospite/pre/durante/post) e giorno di corso da due date
percorso/allenamenti.ts	catalogo dei 18 allenamenti, sblocchi, gruppi per fase
percorso/batteria.ts	carica dalla padronanza media degli obbligatori
percorso/agenda.ts	appuntamenti, corsi, obiettivi, programmi (dato statico)
percorso/percorsoRepo.ts	PercorsoRepo su file locale; in testa, l'implementazione remota che lo sostituirebbe
contenuti/contenuti.ts	video e letture: catalogo, filtri, ricerca, stato
flashcard/flashcard.ts	lingue, mazzo, intervalli fra le ripetizioni
shared/errorMessages.ts	errori tecnici → messaggi italiani, per categoria
shared/retry.ts	backoff esponenziale sui soli errori transitori
shared/fileUtils.ts	estensione da nome/mime, riconoscimento immagini
shared/giorni.ts	mezzanotte locale e inizio giornata: le due primitive di data
shared/idLocale.ts	identificatori generati sul dispositivo per la coda
shared/account.ts	garantisce che l'accesso sia agganciato a un account
controller/ — stato e orchestrazione	
AuthContext.tsx	istanza unica di useAuth per tutta l'app
RipassiContext.tsx	istanza unica di ripassi + cache (una sola subscription Realtime)
avvisoErrore.ts	mostraErrore : traduce, registra su Sentry, mostra. Unico canale
useConnettivita.ts	stato online/offline (NetInfo) per la View
useLocalCache.ts	rotazione all'apertura (max 1/giorno) ed esito dell'ultima
auth/useAuth.ts	sessione, login Google (PKCE) / email, logout; compone useDriveAuth
auth/useDriveAuth.ts	autorizzazione Drive: unico proprietario dello stato
auth/oauthLogin.ts	helper senza stato del login Google (redirect, messaggi, attese)
auth/useAccountDrive.ts	account e cartella Drive, caricati su richiesta
ripassi/useRipassi.ts	stato ripassi, CRUD, politica di retry, Realtime
ripassi/useFormRipasso.ts	stato del form e orchestrazione del salvataggio
allegati/useAllegati.ts	upload a lotti, rinomina, riordino, eliminazione, apertura
allegati/fileDispositivo.ts	picker nativi e apertura file locali (senza stato React)
PercorsoContext.tsx	fase, giorno, padronanze, batteria: letti una volta, condivisi
useCoda.ts	coda di uscita: cosa manca al server, quando drena
useRitento.ts	“la connessione fa i capricci: riprovo...”
useNotificheRipassi.ts	tiene i promemoria locali allineati alle occorrenze
view/ — interfaccia	
theme/theme.ts	palette blu/giallo, rampe 100–900, spaziature, raggi, caratteri: isolata
theme/icone.tsx	il set di icone, per nome di dominio
lib/format.ts	date in italiano, etichette Oggi/Domani/Ieri
lib/calendarUtils.ts	griglia del mese (puro, settimana da lunedì)
screens/GuscioScreen.tsx	il guscio: quattro schede, testata, menu laterale
screens/HomeScreen.tsx	la Home, in quattro varianti secondo la fase
screens/RipassiScreen.tsx	TurboRipassi: ricerca, gruppi per scadenza, storico
screens/*	Login, FormRipasso, DettaglioAllegati, Allenati, Allenamento, Contenuti, Flashcard, Profilo, PercorsoScreens (Programma, Appuntamenti, Corsi, Obiettivi)
components/organic/	il sistema di design, per famiglia: toni , testo , superfici , controlli , indicatori
components/TabBar.tsx	la barra flottante delle quattro schede

Capitolo 7

Modello dati

Tre tabelle di contenuto su Postgres (Supabase), tutte con RLS attiva, più due tabelle di identità e una tabella solo locale sul dispositivo. I binari degli allegati non sono in nessuna di esse: stanno su Google Drive, e `allegati.storage_path` ne conserva l'identificativo.

I dati appartengono all'`account` — la persona — e non alla riga di `auth.users`, che rappresenta un singolo *modo di accedere*. Un account ha N `identita`: email e password, Google, e quante se ne aggiungeranno. È ciò che fa sì che la stessa persona ritrovi gli stessi ripassi qualunque pulsante di login preme.

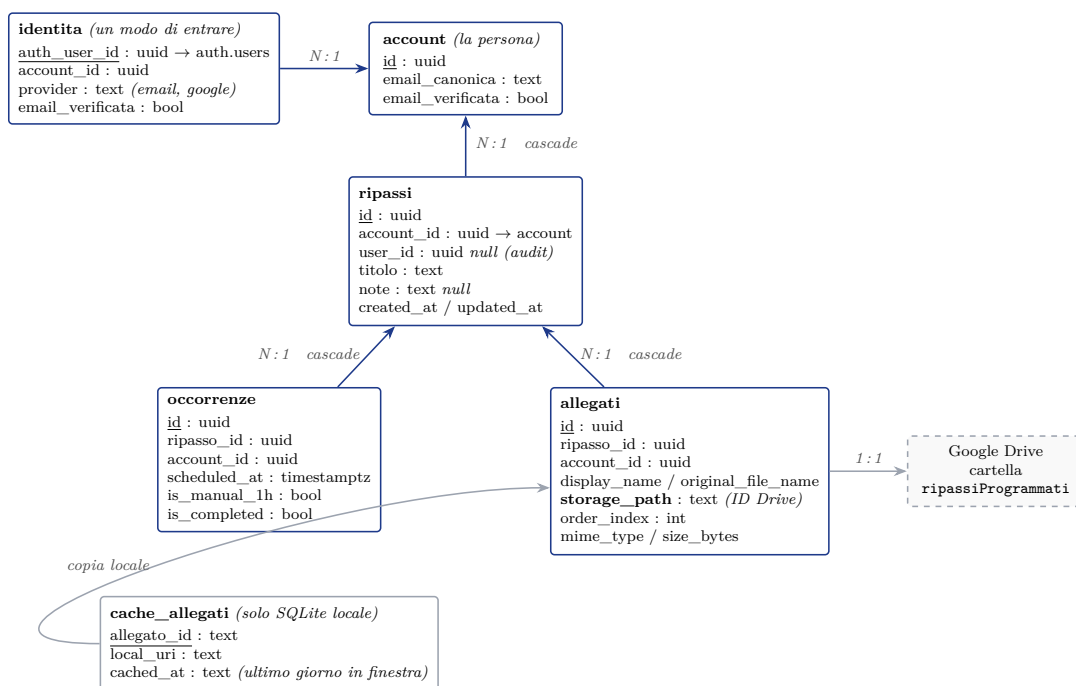


Figura 7.1: Le tabelle Postgres (RLS attiva su tutte), la tabella di sola cache sul dispositivo, e il legame con il file su Drive. Il tratteggio segna ciò che non vive su Postgres. Da notare la direzione delle frecce in alto: i ripassi pendono dall'`account`, non dall'identità con cui si è entrati.

7.1 ripassi

Campo	Tipo	Note
id	uuid	PK, <code>gen_random_uuid()</code>
account_id	uuid	FK <code>account</code> : il proprietario. Default <code>account_corrente()</code>
user_id	uuid	FK <code>auth.users</code> <i>null</i> , default <code>auth.uid()</code> : solo audit
titolo	text	
note	text	nullable, testo semplice
created_at / updated_at	timestampz	<code>updated_at</code> mantenuto da trigger

7.2 occorrenze

Campo	Tipo	Note
id	uuid	PK
ripasso_id	uuid	FK → ripassi, ON DELETE CASCADE
account_id	uuid	proprietario; ridondante ma necessario per RLS efficiente
user_id	uuid	<i>null</i> , solo audit
scheduled_at	timestampz	
is_manual_1h	boolean	true solo per l'occorrenza +1 ora
is_completed	boolean	default false
created_at / updated_at	timestampz	

Alla creazione di un ripasso vengono inserite le occorrenze a **+1 giorno**, **+1 settimana**, **+1 mese**, **+6 mesi** (`occorrenzeDates.ts`); l'occorrenza **+1 ora** solo se l'interruttore è attivo. Nota documentata: il 31 gennaio +1 mese trabocca su marzo (comportamento standard di JavaScript, coperto da test).

7.3 allegati

Campo	Tipo	Note
id	uuid	PK
ripasso_id	uuid	FK → ripassi, cascade
account_id	uuid	proprietario, per RLS
user_id	uuid	<i>null</i> , solo audit
display_name	text	modificabile dall'utente
original_file_name	text	nome originale sul dispositivo
storage_path	text	ID del file su Google Drive (vedi cap. 6)
order_index	integer	riordino manuale
mime_type / size_bytes	text / bigint	

7.4 cache_allegati (solo SQLite on-device)

Campo	Tipo	Note
allegato_id	TEXT	PK
local_uri	TEXT	percorso nel filesystem del dispositivo
cached_at	TEXT	ultimo giorno (locale) in cui era in finestra

Nota su `cached_at`. Il campo è rimasto, ma il suo significato è cambiato e non è più un criterio di decisione: la rotazione elimina per *appartenenza alla finestra* (vedi cap. 8), non per data di download. Oggi `cached_at` è solo un'informazione diagnostica: “l'ultimo giorno in cui questo allegato serviva”. Il criterio originale, ancora descritto nella specifica iniziale, cancellava e ricaricava di continuo file perfettamente validi.

Capitolo 8

Sincronizzazione cross-device

- Ogni scrittura passa da Supabase; ogni dispositivo mantiene **una sola** subscription Realtime (in `RipassiContext`) sulle tre tabelle: alla ricezione di un evento il Controller ricarica e la View si ri-renderizza.
- Gli eventi ravvicinati collassano in un solo ricaricamento (finestra di 200 ms). Senza, ogni mutazione ne produceva due — quello esplicito e l'eco del proprio evento Realtime — e un caricamento a lotti di N allegati ne produceva $N + 1$, ciascuno una rilettura completa di ripassi, occorrenze e allegati con conseguente ri-render della lista, proprio mentre l'app era occupata a comprimere e caricare foto.
- Un contatore di sequenza scarta le risposte fuori ordine: con più ricaricamenti in volo la risposta più vecchia poteva arrivare per ultima e rimettere in lista uno stato già superato. Si manifesta quando la rete è lenta, cioè quando gli eventi si accodano.
- Conflitti: *last-write-wins* su `updated_at` (trigger server-side). Sufficiente per un singolo utente su più dispositivi.
- Assunzione dichiarata: creare/modificare richiede connessione; la cache locale serve alla lettura offline. La scrittura offline (outbox) è fuori MVP.

Capitolo 9

Cache locale e rotazione

Finestra mantenuta in locale: **ieri, oggi, domani** — calcolata sui *giorni locali del dispositivo*, non UTC (un ripasso all'1:00 di notte conta per il giorno giusto).

Ad ogni apertura dell'app (al massimo una volta al giorno per sessione):

1. si scaricano gli allegati dei ripassi con almeno un'occorrenza in finestra (se già presenti si aggiorna solo `cached_at`);
2. si eliminano file e righe di cache degli allegati **non più appartenenti** alla finestra corrente — inclusi quelli eliminati da remoto.



Figura 9.1: La rotazione, al massimo una volta al giorno per sessione. Il dato remoto su Drive non viene mai toccato.

Decisione rilevante emersa dall'audit: l'eliminazione avviene per *appartenenza alla finestra* e non per data di download (`cached_at`); il criterio originale cancellava e ricaricava di continuo i file ancora validi ma scaricati due o più giorni prima (regressione coperta da test). Il dato remoto non viene mai toccato dalla rotazione. Al logout la cache viene svuotata (i file appartengono all'utente).

9.1 Un fallimento silenzioso non è più silenzioso

Un singolo download che fallisce non deve fermare gli altri, quindi il ciclo li racchiude uno per uno. Fino alla revisione precedente però l'errore veniva semplicemente scartato: nessun messaggio, nessun conteggio, nessun evento su Sentry. La conseguenza era la peggiore possibile

per questa funzionalità — token Drive revocato, file cancellato dall'utente dal proprio Drive, spazio esaurito sul dispositivo: in tutti questi casi la cache non si riempiva mai e **nessuno lo sapeva**. L'utente lo scopriva in treno, senza rete, davanti a un allegato che non si apriva; e il crash reporting, integrato proprio per “sapere che qualcosa è andato storto”, non riceveva nulla perché nessun crash era avvenuto.

Ora la rotazione restituisce un esito (**disponibili, falliti**): la Home mostra una riga quando parte del materiale non è disponibile offline, e i fallimenti *anomali* generano un evento Sentry per rotazione (uno, non uno per file). L'assenza di rete è esclusa di proposito: è un esito previsto, e segnalarla riempirebbe la dashboard di eventi su cui non si può agire.

Capitolo 10

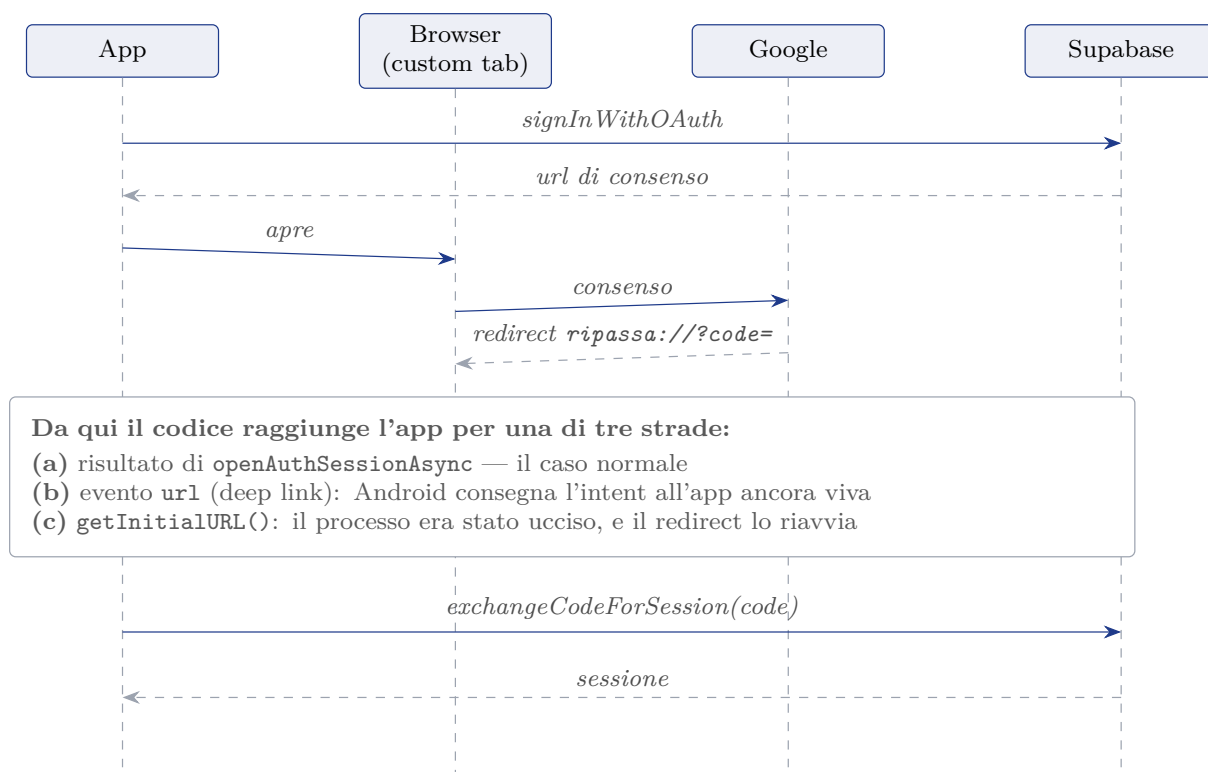
Sicurezza

10.1 Autenticazione

- **OAuth Google con PKCE** (non implicit flow): il redirect `ripassa://` trasporta solo un `?code=` opaco, scambiato per la sessione con un *code verifier* noto solo al client. Un URL di redirect intercettato o loggato non è sufficiente a rubare la sessione.
- **Sessione cifrata a riposo**: JWT e refresh token in `expo-secure-store` (Keychain iOS / Keystore Android), con spezzettamento in voci da ≤ 2 KB (limite di SecureStore). Prima erano in AsyncStorage, in chiaro sul filesystem.
- **Refresh in foreground**: listener AppState che riavvia l'auto-refresh del token quando l'app torna attiva (pattern raccomandato Supabase); elimina i re-login forzati.
- Solo la chiave *publishable* è nel client; la chiave *secret* non compare da nessuna parte nel progetto.

Il redirect torna per due strade, e a volte a processo morto

È la parte più difficile del sistema e merita un disegno. Il redirect che chiude il login può raggiungere l'app in tre modi diversi, e per due di essi il codice deve essere pronto a non essere l'unico ad averlo ricevuto.



Il codice è monouso, e (a) e (b) possono arrivare entrambi in ordine non garantito: `codiciUsati` riconosce il secondo e gli fa attendere l'esito del primo, invece di bruciare il codice una seconda volta.

Figura 10.1: Login Google con PKCE. Il ramo (c) è quello che rendeva il difetto invisibile: la promise che aspettava il browser moriva col processo, e l'app riapriva la schermata di login come se non fosse mai successo niente.

L'autorizzazione a Drive percorre la stessa struttura con una differenza: il *code verifier* PKCE viveva solo dentro l'oggetto `AuthRequest`, e quindi non sopravviveva alla morte del processo. Ora viene scritto in `SecureStore` prima di aprire il browser, insieme allo `state` che fa da guardia CSRF sul ramo (c), dove non c'è più nessun `AuthRequest` a controllarlo.

10.2 Account e identità di accesso

L'isolamento è per **account**, non per riga di `auth.users`. La distinzione non è formale: un utente di `auth.users` è *un modo di accedere*, non una persona. Registrarsi con email e password e poi entrare con Google — stesso indirizzo — produce due righe distinte, perché l'identità è la coppia (`provider`, `subject`) e l'email ne è solo un attributo. Con la proprietà agganciata a `auth.users.id` la stessa persona si ritrovava due insiemi di ripassi disgiunti.

Da qui due tabelle al posto di una: **account** (la persona, proprietaria dei dati) e **identità** (un modo di entrare, N per account, uno per riga di `auth.users`). Qualunque identità risolve allo stesso account.

Un'identità entra in un account esistente **solo a email verificata da entrambe le parti**. Senza il vincolo il modello sarebbe peggiore del difetto che risolve: chi registra in anticipo l'indirizzo di un altro con una password scelta da sé si vedrebbe consegnare l'account al primo accesso Google della vittima. Per questo le identità **email** non si collegano mai alla creazione, ma solo quando il link di conferma viene davvero cliccato. L'invariante è imposta dal database

— un indice unico parziale garantisce al massimo un account *verificato* per indirizzo — non dal codice applicativo. Dettagli e casi limite in `docs/account-identita.md`.

10.3 Row Level Security

Tutte le tabelle hanno RLS attiva con isolamento per account. Su `occorrenze` e `allegati` la policy verifica sia `account_id` sulla riga, sia — difesa in profondità — che il ripasso genitore appartenga allo stesso account:

```
create policy occorrenze_owner on public.occorrenze
for all using (
  account_id = public.account_corrente()
  and exists (select 1 from public.ripassi r
              where r.id = ripasso_id
                  and r.account_id = public.account_corrente())
) with check ( -- identico, piu' user_id is null or = auth.uid() --);
```

`account_corrente()` traduce la sessione nel suo account. È *stable*, quindi Postgres la valuta una volta per statement e non una per riga, e *security definer*, perché altrimenti la RLS su `identita` avrebbe bisogno di questa stessa funzione per decidere se la riga è leggibile.

10.4 I binari degli allegati

Non sono su Supabase e quindi non sono soggetti a RLS: stanno su Google Drive. Le garanzie sono di altra natura e vanno dette per quello che sono:

- **Scope minimo:** `drive.file`. L'app vede *soltanto* i file che ha creato lei, non il resto del Drive dell'utente. Anche la lettura dell'account (`about.get`) resta dentro questo scope: chiedere `userinfo.email` solo per mostrare un indirizzo avrebbe allargato i permessi.
- **Token separati e cifrati a riposo:** `access` e `refresh` token Drive stanno in `expo-secure-store`, non vengono mai inviati a Supabase e vengono cancellati al `logout`.
- **Nessun URL pubblico:** con `drive.file` i file non sono raggiungibili da un link. Per mostrarli l'app li scarica in locale con il token, il che elimina del tutto la superficie “URL firmato che circola”.
- **Confine di responsabilità:** dei binari risponde Google, dei metadati Postgres con RLS. Il prezzo, dichiarato, è che l'utente può cancellare i file dal proprio Drive senza che l'app lo sappia — la rotazione della cache lo scopre e ora lo segnala (cap. 8).

Riordino atomico. Il riordino degli allegati passa dalla funzione Postgres `riordina_allegati`, dichiarata in `SECURITY INVOKER` e quindi soggetta alla stessa policy `allegati_owner`: un utente non può riordinare righe che non gli appartengono nemmeno passando per la funzione.

Permessi richiesti sul dispositivo. Solo la fotocamera (`NSCameraUsageDescription`, `CAMERA`) e la galleria (`NSPhotoLibraryUsageDescription`), cioè i due picker effettivamente usati. Il permesso `RECORD_AUDIO`, ereditato per copia da una configurazione di esempio, è stato rimosso: l'app non ha nessuna funzione audio, e chiedere il microfono a un'app di ripassi è esattamente il tipo di richiesta che il principio del minimo privilegio esiste per evitare.

Alla revisione successiva è emerso che il paragrafo era incompleto: `app.json` dichiarava anche `READ_EXTERNAL_STORAGE`, un secondo residuo dello stesso tipo. Da Android 13 quel permesso è deprecato e non dà più accesso ai media, e i picker usati qui non ne hanno comunque bisogno

— `expo-image-picker` passa dal photo picker di sistema e `expo-document-picker` da SAF, e entrambi restituiscono URI già autorizzati. È stato rimosso: l'elenco sopra è ora quello effettivo.

Capitolo 11

Dove vivono gli allegati

È lo scostamento più vistoso rispetto alla specifica iniziale, ed è deliberato. La sezione 2 della specifica poneva fra le decisioni vincolanti “nessuna dipendenza da iCloud o Google Drive per lo storage applicativo”; oggi i binari degli allegati stanno proprio sul Google Drive dell’utente. Questo capitolo esiste perché una decisione vincolante ribaltata senza spiegazione è indistinguibile da un requisito disatteso.

11.1 Il vincolo che ha forzato la scelta

Il piano gratuito Supabase dà 1 GB di file storage. La specifica lo aveva già individuato come “l’unico vincolo che collide con *nessun limite di dimensione*” e stimava il superamento in qualche mese di uso reale con foto e PDF di studio. Le alternative valutate:

Opzione	Costo	Perché sì / no
Restare su Supabase Storage	gratis fino a 1 GB	Il tetto arriva, e arriva presto. Superarlo significa perdere l’accesso ai propri appunti.
Supabase Pro	25 \$/mese	Fuori dal budget dichiarato (~2 €/mese) di un ordine di grandezza.
Compressione più aggressiva	gratis	Sposta il problema di qualche mese e peggiora la leggibilità di appunti fotografati, che è il contenuto principale.
Google Drive dell’utente	gratis (15 GB)	Scelta. Quindici volte lo spazio, sullo stesso account Google già usato per il login, senza nuovi servizi da gestire.

11.2 Perché non riporta il problema di partenza

L’obiezione ovvia: l’app nasce per risolvere la frammentazione di “Genio in 21 Giorni”, che su iPad usava iCloud e su Android Google Drive. La distinzione è che il difetto non era *usare Drive*, era usare **archivi diversi su piattaforme diverse**. Un unico Drive, raggiunto con le stesse credenziali da Android, iPad e Mac, non frammenta nulla: è un solo archivio con un solo account, esattamente ciò che la decisione vincolante voleva ottenere. Restano su Supabase tutti i metadati — titoli, note, date, ordine, nomi — quindi la sincronizzazione in tempo reale e la logica di dominio non cambiano di una riga.

11.3 Cosa cambia, in concreto

- **Un secondo consenso OAuth**, distinto dal login. L'utente lo concede la prima volta che allega un file, non all'avvio. È il costo più alto della scelta: un secondo flusso di autorizzazione con i suoi modi di fallire, che si porta dietro `driveAuth.ts`, `useDriveAuth.ts` e due regole dedicate nella traduzione degli errori.
- **Lo spazio è quello dell'utente**, non del progetto: non c'è più un tetto condiviso da sorvegliare. La compressione resta comunque attiva, per rispetto dei 15 GB altrui.
- **L'utente può cancellare i file dal proprio Drive** senza che l'app lo sappia. La rotazione della cache se ne accorge e ora lo segnala invece di ignorarlo (cap. 8).
- **Serve una build installata**: il redirect OAuth di Google per i client nativi dev'essere `<applicationId>:/oauthredirect`, che Expo Go non può registrare.
- **Il campo si chiama ancora `storage_path`**. Contiene un ID Drive, non un percorso. Il nome è rimasto per non imporre una migrazione dello schema a un dato che è comunque opaco; il tipo lo documenta esplicitamente.

Capitolo 12

Ciclo di vita di un allegato

12.1 Caricamento

Due scritture su due sistemi che non condividono nessuna transazione, quindi la parte interessante è la compensazione.

1. Se è un'immagine viene compressa (1600px sul lato lungo, JPEG 70%). Un PDF passa invariato.
2. **Upload su Drive in due passi:** prima i metadati (crea un file vuoto nella cartella `ripassiProgrammati` e restituisce l'ID), poi il contenuto binario via `FileSystem.uploadAsync`. Due passi e non uno per non caricare l'intero file in memoria come base64: una foto compressa resta comunque di centinaia di KB, e la conversione era pura spesa. Se il secondo passo fallisce, il file vuoto appena creato viene cancellato.
3. **Riga in allegati** con l'ID Drive in `storage_path`. Se l'insert fallisce, il file su Drive viene cancellato: un binario senza metadati sarebbe spazzatura invisibile nel Drive dell'utente.
4. `size_bytes` registra la dimensione del file *caricato*, non quella dichiarata dal picker. Erano due numeri diversi: le immagini vengono compresse, e salvare l'originale sovrastimava il consumo di circa un ordine di grandezza — proprio sulla classe di file che domina il volume, e proprio mentre il consumo di spazio è la cosa da tenere d'occhio.

Il caricamento **non viene mai ritentato**: crea un file nuovo e una riga nuova, quindi non è idempotente (cap. 13.3).

12.2 Apertura

Un allegato si apre sempre da un file *locale*: con lo scope `drive.file` non esiste un URL pubblico da passare a un viewer.

1. Se è nella cache della finestra, si usa quella copia (dopo aver verificato che il file esista ancora davvero sul dispositivo).
2. Altrimenti si scarica in un file temporaneo nella cache di sistema. Questo download è idempotente, quindi *viene* ritentato.
3. Le immagini si mostrano in un viewer in-app a schermo intero; PDF e altri formati vanno al viewer di sistema (intent `VIEW` con `content://` su Android, share sheet / Quick Look su iOS).

Mai un `file://` verso WebBrowser: su Android è una `FileUriExposedException`.

Ciclo di vita dei temporanei. Il punto 2 aveva un difetto che la prima stesura di questo capitolo non nominava: quei file non venivano rimossi da niente. Il nome deriva dall'id dell'allegato, quindi riaprire lo *stesso* allegato sovrascrive, ma ogni allegato *distinto* aperto fuori finestra — cioè la normale consultazione dello storico — lasciava dietro di sé un PDF o una foto per sempre. La cartella di cache di sistema viene potata dal sistema operativo solo sotto pressione di spazio, e su Android quel momento tende ad arrivare ben dopo che l'utente si è accorto dell'app che occupa centinaia di MB. In un progetto che ha eletto il consumo di spazio a vincolo dimensionante (cap. 11) era una falla di categoria.

Ora i temporanei stanno in una sottocartella dedicata (`allegati-tmp/`), il che rende la potatura l'elenco di una cartella che non contiene altro invece di un filtro su una convenzione di nomi. La politica è a tempo: si conservano sette giorni dall'ultimo uso, la potatura gira insieme alla rotazione della cache (al massimo una volta al giorno, cadenza adatta a una politica misurata in giorni) e al logout si cancella tutto, perché a quel punto quei file non riguardano più nessuno. La selezione è una funzione pura in `cacheLogic.ts`, testata: un file di cui non si riesce a leggere la data conta come scaduto, perché non poterlo dimostrare recente e tenerlo per sempre è esattamente l'esito da evitare. Il costo di un errore di stima è un download in più.

12.3 Rinomina, riordino, eliminazione

Rinomina e riordino toccano solo i metadati. Il riordino è una singola chiamata transazionale (`riordina_allegati`): la versione precedente mandava un UPDATE per riga in parallelo, e un fallimento a metà lasciava `order_index` duplicati o con buchi, senza modo di tornare indietro. L'eliminazione rimuove la riga, poi il file su Drive, poi la copia in cache — in quest'ordine: cancellare prima il binario lascerebbe una riga che punta a un file inesistente, cioè un allegato che non si apre più.

La conseguenza dell'ordine giusto è però che il fallimento del secondo passo produce un orfano *irrecuperabile*: la riga che conteneva l'id Drive non esiste più, quindi né l'app né l'utente potranno mai risalire a quale file nella cartella sia spazzatura, e lo spazio resta occupato per sempre. Il codice lo ignorava in silenzio, con un `catch` vuoto, nello stesso file che venti righe più su tratta l'orfano opposto (binario senza metadati) come inaccettabile e implementa la compensazione. Ora `elimina` restituisce un esito: l'operazione riesce lo stesso, perché per l'utente l'allegato è sparito davvero e un avviso sarebbe rumore, ma l'esito riporta l'id rimasto orfano e il Controller genera l'evento Sentry corrispondente. Stesso trattamento per il rollback fallito del caricamento, che lasciava un orfano speculare.

Capitolo 13

Affidabilità in rete

Tre problemi emersi rileggendo il codice in vista di una distribuzione ad utenti reali, e come sono stati chiusi.

13.1 Errori comprensibili

Gli errori di Supabase/Postgres sono in inglese e citano dettagli interni (`duplicate key value violates unique constraint "ripassi_pkey"`). Mostrarli in un `Alert` a un utente italiano è inaccettabile. `errorMessages.ts` li classifica per categoria (rete, autenticazione, permessi, duplicato, non trovato, spazio) e restituisce titolo e messaggio in italiano, con un flag `ritentabile`.

La mappatura è una **tabella di regole**, non una catena di `if`: le regole sono dati, il confronto è un solo ciclo, e aggiungere un caso significa aggiungere una riga invece di trovare il punto giusto in una funzione di 150 righe.

La proprietà critica è l'ordine. Le regole si applicano dall'alto in basso, quindi ciò che tiene corretto il modulo non è “esiste la regola giusta” ma “nessuna regola più larga arriva prima”. Due difetti reali nascevano proprio lì, e sono stati chiusi rendendo strutturale ciò che era testuale:

- **Codici di stato.** Erano cercati come sottostringhe ("`500`", "`404`", "`413`") dentro un testo che include il corpo della risposta di Google. Un errore `403` il cui JSON citava una quota di “500 richieste” veniva classificato guasto transitorio del server, e quindi *ritentato* tre volte con backoff contro una richiesta che non poteva riuscire. Ora gli errori Drive trasportano lo stato come campo (`DriveHttpError.status`) e le regole confrontano un numero.
- **Il frammento "`session`".** catturava qualunque messaggio di `expo-auth-session`: uno schema di redirect non registrato diceva all'utente “la sessione è scaduta, esci e accedi di nuovo”, cioè l'unica azione che non poteva servire a niente. Ora i frammenti sono espliciti (`session expired`, `auth session missing`, ...).

I test verificano la categoria, che il testo tecnico *non* compaia nel messaggio finale, e — con un blocco dedicato di casi negativi — che nessuna regola larga preceda quelle specifiche.

L'unica eccezione, e perché esiste. Il messaggio tradotto è sempre la voce principale, ma quando la traduzione *non* riesce a classificare l'errore (categoria `sconosciuto`) il testo originale viene allegato, troncato, sotto la voce “Dettagli”. Il motivo è pratico: un caricamento di allegato può fallire su Drive, su Postgres o sul filesystem, e “Operazione non riuscita” li rende indistinguibili — l'utente non ha niente da riferire e chi legge il crash report nemmeno. Vale in tre punti: `mostraErrore` per la categoria sconosciuta, e i due messaggi del login Google,

che indicano anche in quale metà del flusso si è rotto e quale redirect era atteso. Un errore *riconosciuto* resta invece pulito: il dettaglio tecnico non lo tocca mai.

13.2 Offline esplicito

Senza rilevamento della connettività, “sei offline” e “è successo un errore” finivano nello stesso alert generico. `useConnettivita` incapsula `NetInfo` dietro il Controller e restituisce un solo booleano; Home e Login mostrano un banner dedicato. Solo una lettura esplicitamente negativa conta come offline, così una sonda di raggiungibilità lenta non produce un falso allarme.

13.3 Retry: dove sì e dove no

`retry.ts` ritenta con backoff esponenziale, ma **solo** quando l’errore è transitorio (rete, 5xx) — ritentare un errore di vincolo o di permesso ripete solo lo stesso fallimento — e **solo** su operazioni idempotenti.

Operazione	Retry	Perché
lettura elenco	sì	idempotente
modifica / completa / sposta	sì	stesso input, stesso stato finale
elimina (ripasso, allegato)	sì	idempotente
rinomina / riordina allegati	sì	idempotente (il riordino è transazionale)
download per l’apertura	sì	idempotente, e l’utente sta aspettando
crea ripasso	no	insert non idempotente
carica allegato	no	crea file Drive + riga nuovi
download in rotazione cache	no	scelta diversa: vedi sotto

Le esclusioni sono deliberate e sono il punto meno ovvio del capitolo.

Le prime due per non idempotenza: un errore di rete può significare “la richiesta è arrivata ma la risposta si è persa”, e ritentare creerebbe un secondo ripasso, o un secondo file su Drive con una seconda riga in `allegati` (il nome su Drive è generato con `Date.now()`, quindi ogni tentativo produce un file diverso). Chiedere all’utente di ripremere è meno grave di un duplicato silenzioso.

La terza per un motivo opposto: il download *sarebbe* idempotente, ed è infatti ritentato quando l’utente apre un allegato. Ma la rotazione della cache può riguardare decine di file in una volta, all’avvio, tipicamente proprio quando la rete è cattiva: tre tentativi con backoff su ciascuno bloccherebbero l’app per minuti nel momento peggiore. La rotazione riprova alla successiva apertura, che è la stessa cosa senza l’attesa. Ciò che invece non deve più succedere è che il fallimento passi inosservato: vedi il capitolo sulla cache.

13.4 Ordinamento totale

La classificazione attivi/storico viveva in `HomeScreen` e ordinava con un fallback ?? "" che, su una data assente, produce `new Date("").getTime() = NaN` e rende il comparatore inconsistente. Nel flusso attuale quel ramo non era raggiungibile (il filtro `isStorico` garantisce che gli attivi abbiano sempre un’occorrenza futura), ma la correttezza dipendeva da un invariante non locale, cioè dalla coerenza fra due funzioni separate. La logica è stata spostata in `ripassiLogic.ts` con una chiave di ordinamento totale (mai NaN, con fallback su `created_at`) e un tie-break sull’id, così due ripassi con la stessa data non si scambiano di posto fra un ricaricamento e l’altro.

Capitolo 14

Crash reporting e resilienza

Prima di distribuire build a utenti reali servivano due cose che l'MVP non aveva: sapere *che* un crash è avvenuto, ed evitare che un errore di render lasci lo schermo bianco.

- **Sentry** (`@sentry/react-native`), incapsulato in `src/config/crashReporting.ts`. Vale la stessa regola del client Supabase: *nessun altro modulo importa l'SDK*, si passa da `initCrashReporting()` e `reportError()`. Il DSN si legge con la stessa priorità `env` → `app.json/extra` usata altrove (`EXPO_PUBLIC_SENTRY_DSN`).
- **Degrado morbido**: senza DSN il modulo fa no-op e logga un warning; l'app parte e builda normalmente. Il reporting è inoltre disattivato in `__DEV__`, per non mescolare errori di sviluppo e crash di produzione. `tracesSampleRate` è 0: qui interessano i crash, non il performance tracing.
- **ErrorBoundary** (`src/view/components/ErrorBoundary.tsx`), avvolge l'intero albero in `App.tsx`. Cattura gli errori di render/lifecycle, li inoltra a Sentry con lo stack dei componenti e mostra un fallback a tema con un pulsante “Riprova” che resetta lo stato, invece di un crash silenzioso. Deve essere una classe: React non espone gli error boundary come hook.
- **Sentry.wrap** sull'export di `App.tsx`, per intercettare anche ciò che sta fuori dal render tree (crash nativi, errori JS non gestiti).

Il config plugin `@sentry/react-native` è registrato in `app.json`: le build native lo includono senza passaggi manuali. Resta da registrare `EXPO_PUBLIC_SENTRY_DSN` su EAS prima della distribuzione (procedura in `BUILD.md`).

Capitolo 15

Test e qualità

```
npm run verifica    # lint + typecheck + test, in quest'ordine
npm run lint        # ESLint: regole Expo + confini fra i livelli
npm run typecheck   # tsc --noEmit (strict)
npm test            # 795 test su 63 suite (jest-expo)
npm run test:coverage
```

Gli stessi comandi girano in integrazione continua (`.github/workflows/ci.yml`) a ogni push e su ogni pull request. Nessuno tocca la rete: la suite copre logica pura e moduli il cui I/O è simulato, quindi il job è autosufficiente e non richiede segreti. La CI esegue la suite *una volta sola*, con la copertura: prima la esegue due volte, una in più che non aggiungeva nessuna informazione.

15.1 La seconda suite: i test SQL

Esiste una seconda suite che il capitolo precedente non nominava, e che vale la pena nominare perché copre la parte del sistema con le conseguenze più severe: `supabase/tests/` verifica su righe vere il modello account/identità e le policy RLS — cioè il meccanismo che impedisce a un account di leggere i ripassi di un altro, e il vincolo di email verificata che impedisce a un indirizzo occupato di catturare l'account della vittima. Sono proprietà che non si vedono leggendo una definizione di tabella: vanno provocate.

```
npm run test:db      # richiede SUPABASE_DB_URL
```

Non girano in integrazione continua, e la ragione è dichiarata: sono scritti contro lo schema `auth` di un progetto Supabase reale, non contro un Postgres nudo, e terminano in `rollback` in modo da non lasciare traccia. Portarli in CI richiederebbe lo stack Supabase locale completo. Vanno eseguiti a mano dopo ogni modifica allo schema o alle policy: un test che non gira è documentazione, non verifica, e questo è il limite più serio che resta nella dimensione dei test.

15.2 Copertura per livello

Livello	Soglia	Che cosa è coperto
<code>model</code>	92 %	logica di dominio, repository (Supabase e Drive simulati), cache SQLite, coda di uscita, promemoria, percorso e catalogo del corso, traduzione errori, retry, gestione dei token Drive
<code>config</code>	62 %	risoluzione della configurazione, DSN Sentry, client ID Drive, canale delle notifiche, guardia di init
<code>controller</code>	79 %	gli hook di ripassi, form, allegati, coda, ritento, connettività, cache locale, promemoria, percorso, account Drive e autorizzazione Drive
<code>view</code>	87 %	helper puri (date, calendario), i componenti del sistema di design, il confine d'errore e tutte le schermate
globale	85 %	rete di sicurezza sotto le quattro soglie di livello

La tabella riporta le *soglie* di `package.json` e non le percentuali misurate. La differenza non è formale: una soglia è applicata dalla CI e non può divergere dal codice, una percentuale trascritta a mano invecchia al primo commit — come era già successo al conteggio dei test, che questa relazione riportava con tre valori diversi.

Le soglie sono per livello, e non una sola media globale, perché i quattro livelli hanno rischi diversi. Con la sola soglia globale la copertura alta del `model` faceva da cuscinetto a quella bassa della `view`: si potevano cancellare i test di un hook e restare sopra soglia aggiungendo qualche test di componente. Una media non è una garanzia.

15.3 Che cosa i test verificano davvero

Le asserzioni sono sul comportamento, non sull'implementazione. Alcuni esempi che valgono più dell'elenco completo:

- **Regressione da perdita di dati:** aprendo la scheda di un ripasso prima che la lista sia caricata, un tocco su Salva non deve scrivere campi vuoti sopra titolo e note memorizzati.
- **Non idempotenza:** un errore di rete durante la creazione di un ripasso o il caricamento di un allegato deve produrre *una sola* chiamata, mai due.
- **Idempotenza:** rinomina, riordino ed eliminazione devono invece ritentare, e ricaricare la lista una volta sola.
- **Compensazione:** se l'insert dei metadati fallisce, il file appena creato su Drive dev'essere cancellato; e un rollback fallito non deve mascherare l'errore originale.
- **Ordine delle regole di traduzione:** casi negativi che fissano i confini fra regole (un 403 con “500” nel corpo non è un guasto transitorio; un errore di `expo-auth-session` non è una sessione scaduta).
- **Osservabilità della cache:** un download fallito non ferma gli altri, viene contato, e genera un evento Sentry solo se non è assenza di rete.
- **Ordinamento dei figli:** Postgres restituisce le righe innestate in ordine arbitrario; il repository garantisce occorrenze cronologiche e allegati nell'ordine scelto dall'utente.

15.4 Limiti noti di copertura

Dichiarati, non nascosti:

- I due flussi OAuth restano il punto più scoperto, anche se meno di prima: `useDriveAuth` ha ora una suite propria (17 asserzioni sulle decisioni deterministiche — quando chiedere

il consenso, quando limitarsi a rispondere di no, e l'asimmetria per cui un refresh a vuoto può alzare la bandiera dell'autorizzazione ma non abbassarla, perché offline un refresh torna vuoto anche quando il consenso c'è). `useAuth` ne ha una ferma a circa due terzi delle istruzioni. In entrambi i casi la parte che resta fuori è la stessa: il viaggio attraverso il browser. Sono il codice più delicato del progetto, ma anche quello il cui comportamento dipende da come il sistema operativo consegna un redirect e da quando decide di uccidere il processo: simularlo darebbe fiducia in un modello, non nel dispositivo. Restano verificati a mano, sui casi documentati in `BUILD.md`.

La formulazione precedente si fermava qui, e diceva troppo: era vera per *quando* il redirect arriva e se il processo sopravvive, non per che cosa il codice fa con lo `state` che riceve. La distinzione ora è esplicita, e la seconda metà è coperta: `driveAuth.ts` ha una suite propria su ciò che è deterministico — persistenza del verifier PKCE prima di aprire il browser, guardia CSRF sullo `state`, redirect URI derivato dall'`applicationId`, unicità del codice, e i tre esiti del refresh (valido, revocato, rete caduta). Resta fuori solo il viaggio attraverso il browser.

È stato lo scrivere quei test a far emergere un difetto reale: la guardia CSRF richiedeva che lo `state` fosse presente *sia* nel redirect *sia* nel record salvato, quindi un redirect che semplicemente non ne portava nessuno saltava il controllo — cioè esattamente ciò che farebbe chi sceglie che cosa mandare. Ora la condizione guarda solo lo `state` salvato.

- Il fallback della configurazione su `app.json/extra` non è coperto: `jest-expo` sostituisce `expo-constants` a livello di modulo nativo, quindi sotto `jest Constants.expoConfig` è `undefined` e non è sovrascrivibile con `jest.mock`. Il ramo è esercitato nelle build reali; il limite è annotato anche nel file di test.
- `percorsoRepo` è l'unica copia dello stato del percorso e vive su file, non sul server: i test coprono i casi difensivi (file assente, troncato, campi non validi, disco pieno), non un ciclo di vita reale su dispositivo, che non esiste finché non esiste l'API.
- `useConnettivita`, `useLocalCache`, `useNotificheRipassi` e `useAccountDrive` erano a copertura nulla e ora hanno una suite ciascuno. La motivazione che avevano (“hook sottili sopra un modulo nativo”) si è rivelata sbagliata proprio dove contava: le decisioni interessanti non sono del modulo sottostante ma di questi hook, e sono tutte su *quando* — non chiedere il permesso delle notifiche prima che ci sia qualcosa da ricordare, non far ripartire la rotazione della cache a ogni evento Realtime, non trattare una sonda di rete ancora in corso come un “sei offline”. Nessuna di queste è verificabile in `NetInfo` o nello scheduler di sistema.
- Restano scoperti `fileDispositivo` (selettore di file e fotocamera: è quasi solo chiamate a moduli nativi), `secureAuthStorage` e i due provider `AuthContext/RipassiContext`, che sono involucri sopra hook già coperti. È il residuo dichiarato, non l'assenza di un piano.

15.5 Controlli statici

`tsc` in modalità strict, con `noUnusedLocals` e `noUnusedParameters` attivi: il codice morto non passa. ESLint aggiunge le regole di Expo (fra cui `react-hooks`, che ha fatto emergere due effetti che scrivevano stato durante il render) e le quattro regole di confine fra i livelli descritte nel capitolo sull'architettura.

Un difetto trovato dal linter appena installato. `expo/no-dynamic-env-var` ha segnalato che `env.ts` leggeva `process.env[nome]` con un nome calcolato. La trasformazione Babel di Expo sostituisce `process.env.EXPO_PUBLIC_X` *testualmente* in fase di build: con un accesso dinamico non c'è più nessun oggetto da interrogare a runtime, quindi in una build installata quel ramo restituiva sempre `undefined`. Funzionava tutto lo stesso solo perché i valori sono

anche in `app.json`, ma le variabili registrate su EAS erano peso morto. Ora ogni variabile è scritta per esteso.

Capitolo 16

Build e distribuzione

16.1 Aggiornamento dello schema

`supabase/schema.sql` è idempotente e va rieseguito dopo un aggiornamento del codice: la revisione corrente aggiunge la funzione `riordina_allegati`, senza la quale il riordino degli allegati risponde “operazione non riuscita”. È l’unico passo manuale richiesto lato server.

16.2 Fase 0 — sviluppo (Expo Go)

`npm start` sul Mac, scansione QR con Expo Go (stessa rete Wi-Fi).

Limite di Expo Go: gli allegati non funzionano. Il client OAuth nativo di Google esige il redirect `<applicationId>:/oauthredirect`, che Expo Go non può registrare (lì lo schema diventa `exp://...`, che Google rifiuta). Tutto il resto — login, ripassi, occorrenze, ricerca, sincronizzazione — funziona. Per lavorare sugli allegati serve una development build (`expo-dev-client` è già fra le dipendenze). Il tunnel ngrok si era rivelato inaffidabile e il pacchetto `@expo/ngrok` è stato rimosso dalle dipendenze: era anche l’unica vulnerabilità segnalata da `npm audit` per cui non esisteva una patch a monte.

16.3 Sicurezza delle dipendenze

`npm audit` riporta zero vulnerabilità. Le quattro segnalate in origine (`brace-expansion`, `postcss`, `tar`, `uuid`) erano tutte transitive del solo toolchain di build — Metro, Jest, prebuild — e quindi non presenti nel bundle installato sul telefono. Sono fissate con la sezione `overrides` di `package.json` anziché con `npm audit fix --force`: quel comando avrebbe aggiornato expo alla SDK 57 lasciando però indietro `react-native` e `babel-preset-expo`, producendo un insieme di versioni incoerente e non funzionante. Per `brace-expansion` non esiste una versione corretta nelle linee 1.x e 2.x, per cui l’override è necessariamente globale. Ogni override va rimosso quando Expo aggiorna la dipendenza a monte.

16.4 Fase 1 — app installate (vedi BUILD.md)

- **Android:** `eas build -p android -profile preview` → APK dal cloud Expo (gratis, ~15 build/mese), installazione diretta. Variabili d’ambiente registrate su EAS (il `.env` locale non viene caricato).
- **iPad:** `npx expo prebuild -p ios + npx expo run:ios -device` con Apple ID gratuito (Personal Team). Limite: firma valida 7 giorni, rinnovo = rilanciare il comando con iPad collegato. Richiede Xcode e CocoaPods sul Mac.

- **Mac M2:** stessa app come “My Mac (Designed for iPad)” da Xcode. È il rischio 11.1 della specifica, in corso di validazione.
- Redirect OAuth per le build standalone: `ripassa://` va aggiunto agli URL consentiti in Supabase Auth. La build dimostrativa *TurboGenio* (stesso codice, package e scheme diversi perché possa convivere sul telefono con l’app in uso) richiede allo stesso modo `turbogenio://`: è un’aggiunta, non una sostituzione, e i due redirect restano validi insieme.

Vincolo verificato: percorso senza spazi

La build iOS nativa è stata provata end-to-end (Xcode 26.6). Fallisce se il percorso del progetto contiene spazi — la cartella attuale `Side hustle` rompe lo script `Expo get-app-config-ios.sh`, invocato senza virgolette (`bash: .../Documents/Side: No such file or directory`). Spostando il progetto in un percorso senza spazi la stessa build raggiunge `** BUILD SUCCEEDED **` e produce `Ripassa.app` (binario universale arm64+x86_64). **Il codice compila:** l’unico ostacolo è lo spazio nel percorso. Android via cloud EAS non ne è affetto. Dettagli e comandi in `BUILD.md`.

Capitolo 17

Scostamenti dalla specifica iniziale

`ripassa-app-spec.md` raccoglie i requisiti di luglio 2026 ed è conservato come documento storico. Quattro punti sono stati superati consapevolmente; elencarli è più onesto che riscrivere la specifica come se ci avesse visto giusto dall'inizio.

Specifica	Oggi	Perché
§2: nessuna dipendenza da Google Drive per lo storage	I binari degli allegati stanno sul Drive dell'utente	Il tetto di 1 GB di Supabase, già segnalato dalla specifica stessa come l'unico vincolo critico. Argomentazione completa nel cap. 11.
§7.3: elimina dalla cache le righe con <code>cached_at</code> fuori finestra	Elimina per appartenenza alla finestra	Il criterio originale cancellava e riscaricava file ancora validi solo perché scaricati due giorni prima. Regressione coperta da test.
§4: tre livelli (Model, Controller, View)	Tre livelli più <code>config</code> trasversale	L'infrastruttura (client, SDK, storage cifrato) non appartiene al dominio e serve a tutti. Le dipendenze consentite sono ora verificate dal linter.
§8: Fase 0 su Expo Go come percorso completo	Gli allegati richiedono una build installata	Google esige un redirect OAuth derivato dall' <code>applicationId</code> , che Expo Go non può registrare.
§1: una sola funzione, i ripassi	Quattro schede (Home, Allenati, TurboRipassi, Contenuti) più le schermate del menu laterale	Il redesign del corso ha portato dentro l'app anche allenamenti, contenuti, flashcard e agenda. Sono <i>contorno</i> nel senso preciso che nessuno di essi ha ancora un'API: il catalogo è dato statico nel Model e lo stato personale sta in un file locale (<code>percorsoRepo</code>).
§2: nessuna scrittura senza rete	Coda di uscita su file, con copia privata degli allegati	Un ripasso creato in metropolitana andava perso: era il caso d'uso che motivava la cache offline in lettura.

Restano invece rispettati senza eccezioni: separazione MCV obbligatoria, budget, app nativa, nessuna statistica, nessuna notifica push.

Un'eccezione da dichiarare. “Un solo account per tutti i dispositivi” vale ancora per i ripassi, gli allegati e la loro sincronizzazione — cioè per tutto ciò che ha una tabella su Supabase — ma **non** per lo stato del percorso introdotto dal redesign: iscrizione, padronanze degli allenamenti, lingua delle flashcard e programma scelto stanno in un file locale, e oggi

due dispositivi della stessa persona vedono valori diversi. Non è un limite tecnico ma l'assenza di uno schema: il repository ha già la forma di quelli server — **PercorsoRepo**, interfaccia iniettata nel provider come tutte le altre — e la tabella con la sua policy RLS è già scritta in `supabase/futuro/0004_percorso.sql`, insieme all'implementazione remota che la userebbe. Non è applicata per una decisione di prodotto dichiarata nei prossimi passi: si farà se e solo se il prodotto verrà venduto ufficialmente a Genio in 21 giorni. Finché non c'è, resta lo scostamento più serio rispetto alla promessa di partenza del progetto, ed è scritto qui perché sia il lettore a giudicarlo e non a scoprirlo.

Capitolo 18

Metodo di lavoro e contributo personale

Il progetto dichiara fin dalla specifica che il codice è generato da AI e supervisionato, non scritto a mano. È una scelta di metodo, non una scorciatoia, e come tutte le scelte di metodo va documentata — altrimenti la domanda “e allora cosa hai fatto tu?” resta senza risposta pur avendone una.

18.1 Il ciclo effettivo

1. **Specifica scritta a mano** prima di qualsiasi codice (`ripassa-app-spec.md`): obiettivo, decisioni vincolanti, modello dati, schermate, rischi da validare per primi. È il documento che rende generabile il resto.
2. **Generazione** guidata, un’area funzionale alla volta.
3. **Verifica**, con criteri di accettazione fissi: `npm run verifica` (lint, typecheck, test) deve passare; l’area toccata va provata sul dispositivo reale; le decisioni non ovvie devono risultare dal codice sotto forma di commento che spiega *perché*, non *cosa*.
4. **Correzione guidata** quando la verifica trova qualcosa — che è il punto in cui si concentra il contributo personale, perché quasi nessuno di questi difetti è visibile leggendo il codice.

18.2 Che cosa ha trovato la verifica

Undici esempi. I primi sette sono tracciabili nella storia dei commit; gli ultimi quattro appartengono alla revisione di design, che al momento di questa stesura è ancora su un ramo di lavoro e quindi non ha hash citabili. Sono la prova più concreta di che cosa significhi “supervisione” qui, e vale la pena notare che nessuno degli undici è stato trovato da un test: i test servono a non ripederli.

Commit	Problema	Come è emerso
bf3a755	La cache cancellava e ricaricava di continuo file ancora validi	Rilettura del criterio di eliminazione: la specifica stessa prescriveva la regola sbagliata (<code>cached_at</code> invece di appartenenza)
255149e	Sessione salvata in chiaro su disco; OAuth in implicit flow; RLS senza controllo sul genitore	Audit di sicurezza condotto rileggendo il codice con la domanda “cosa vede chi ottiene il dispositivo?”
2abbe0a	I ripassi di oggi sparivano dalla sezione “Da fare” appena passata l’ora	Uso reale dell’app: un difetto di prodotto, non di codice — il confronto era per istante e doveva essere per giornata
5a8efbc	Un <code>override</code> di sicurezza rompeva la build nativa Android	<code>expo run:android</code> fallito dopo <code>npm audit fix</code> : diagnosi del codegen di React Native e scelta di accettare la vulnerabilità documentandola (è un DoS su pattern glob in build locale)
1581c99	Google rifiutava lo schema <code>ripassa://</code> con Error 400	Prova sul dispositivo: i client nativi esigono <code><applicationId>:/oauthredirect</code>
7b4cf7c	Consenso concesso, ma l’app tornava al login senza errori	Riproduzione sul Samsung: Android uccide il processo mentre il browser è in primo piano, il redirect <i>riavvia</i> l’app, e la promise che aspettava era morta col processo. Diagnosi impossibile da leggere nel codice
ef3c315	Con token Drive revocato l’app diceva “riavvia l’app”, l’unica azione inutile	Test dell’errore reale: <code>isAuthorized()</code> restava vero anche senza token utilizzabile
<i>redesign</i>	Etichetta bianca su pillola gialla nella barra delle schede: la scheda selezionata era illeggibile	Screenshot dall’emulatore dopo il cambio di palette. Nessun test poteva vederlo: le asserzioni sono sui testi, e il testo c’era
<i>redesign</i>	Il pulsante “Aggiungi ripasso” spostato in basso a destra, flottante sopra la lista	Revisione d’uso: era una scelta grafica presa senza chiedersi dove stesse prima. Riportato in alto, nel flusso
<i>redesign</i>	<code>mezzanotteLocale</code> duplicata in <code>fasi.ts</code> e <code>agenda.ts</code> , con i due corpi già divergenti (uno normalizzava l’orario, l’altro no)	Rilettura: il commento della seconda copia diceva “vedi la nota in <code>fasi.ts</code> ”, cioè ammetteva la copia invece di toglierla. Estratte in <code>shared/giorni.ts</code>
<i>redesign</i>	La regola ESLint sul confine della View non vedeva i cataloghi arrivati col redesign	Rilettura della regola, non un fallimento del linter: era scritta come lista di divieti e falliva aperta. Ora è una lista di moduli ammessi

18.3 Le quattro revisioni

1. **MVP** (14fbbb2...eb83eae): funzionalità, prima suite di test, audit di sicurezza, prima documentazione.
2. **Affidabilità e distribuzione** (aebcea6...93787e9): migrazione a Drive, errori traducibili, retry, connettività, crash reporting, e la lunga serie di correzioni sui due flussi OAuth.
3. **Struttura e verifica** (487b9ea in poi): riorganizzazione per dominio dentro i livelli MCV a comportamento invariato, poi introduzione di linter, confini di architettura eseguibili, CI, test su repository e Controller, e riallineamento di questa documentazione al codice. In questa fase sono stati chiusi anche cinque difetti funzionali, fra cui il più grave dell’intero progetto: aprire la scheda di un ripasso prima che la lista fosse caricata e salvare cancellava titolo e note su tutti i dispositivi.
4. **Redesign** (ramo `app-intera`): l’app diventa un compagno del corso (cap. 3), con il sistema di componenti, le quattro fasi e lo stato del percorso persistito. È la revisione con più codice nuovo e, in proporzione, quella in cui il ciclo descritto sopra è stato applicato meno: la specifica era una serie di schermate già disegnate, non un documento scritto prima, e la verifica è stata fatta guardando l’app sull’emulatore invece che sui criteri fissati in anticipo. I quattro difetti in fondo alla tabella qui sopra sono il prezzo di quella differenza — tre su quattro sono cose che si vedono solo guardando, e il quarto è una regola di architettura che era stata scritta e non riletta.

18.4 Limite dichiarato del metodo

Il codice generato è uniformemente buono in ciò che è locale (nomi, struttura di una funzione, gestione di un caso) e sistematicamente debole in ciò che dipende dal comportamento reale del sistema operativo, dalla vita del processo e dall'uso quotidiano. Dieci degli undici difetti elencati sopra appartengono alla seconda categoria — l'unica eccezione è la regola ESLint scritta per enumerazione, che era leggibile sulla pagina e non è stata riletta. È il motivo per cui la verifica sul dispositivo non è sostituibile con i test, e per cui i test servono comunque: intercettano la terza categoria, quella delle regressioni introdotte correggendo le prime due.

Capitolo 19

Rischi aperti

1. Comportamento dei moduli nativi (fotocamera, picker) su macOS in modalità “Designed for iPad” — da testare sul dispositivo reale.
2. Rinnovo settimanale della firma iPad con Apple ID gratuito — fastidio accettabile o no? Decisione dopo l’uso.
3. **Copertura dei due flussi OAuth:** verificati a mano, non da test automatici (le ragioni sono nel cap. 15). È il rischio residuo più alto, perché è anche l’area che storicamente ha richiesto più correzioni.
4. **SHA-1 di release:** il client OAuth Android è registrato con l’impronta del keystore di debug. La prima build EAS di produzione userà un keystore diverso, e il login Drive fallirebbe solo su quella build. Procedura in `BUILD.md`, sottosezione «*SHA-1: debug non è release*» (dentro la sezione 0, Prerequisiti, punto sul client OAuth Android). Allo stesso nodo appartiene il fatto che oggi esista un solo client OAuth, di tipo generico: `app.json` ne dichiarava tre, ma con lo stesso valore copiato tre volte, il che su Google Cloud non può essere corretto (il tipo di un client è fissato alla creazione). Le due chiavi ridondanti sono state tolte; `driveConfig.ts` le legge non appena i client per piattaforma esistono.
5. Consumo dello spazio sul Drive personale con l’uso reale — non è più un tetto condiviso da 1 GB, ma resta da misurare.

Chiusi rispetto alla revisione precedente. Il tetto di 1 GB dello storage Supabase (non più applicabile: cap. 11) e la compilazione iOS, verificata end-to-end fino a `BUILD SUCCEEDED`.